

MODULE 07

Supervised Learning - Classification

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Supervised Learning - Classification

Classification Overview

Classification predicts which category an input belongs to.

Binary: 2 classes (spam/not spam, 0/1)

Multi-class: 3+ classes (cat/dog/bird)

Multi-label: multiple labels per instance

Key algorithms:

Logistic Regression, Decision Trees,

Random Forest, SVM, KNN, Naive Bayes, XGBoost

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print('Accuracy:', model.score(X_test, y_test))
```

Decision Trees and Random Forest

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Decision Tree
dt = DecisionTreeClassifier(max_depth=5, min_samples_split=10)
dt.fit(X_train, y_train)

# Random Forest (ensemble of trees)
rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    min_samples_split=5,
    random_state=42
)
rf.fit(X_train, y_train)

# Feature importance
import pandas as pd
feature_imp = pd.Series(rf.feature_importances_,
```

```
index=feature_names).sort_values(ascending=False)
```

SVM and KNN

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Support Vector Machine
svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
svm.fit(X_train, y_train)

# K-Nearest Neighbors
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)

# Find best k
from sklearn.model_selection import cross_val_score
scores = [cross_val_score(KNeighborsClassifier(k), X, y, cv=5).mean() for k in
range(1, 21)]
best_k = scores.index(max(scores)) + 1
```

Classification Metrics

```
from sklearn.metrics import (
accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score
)

print('Accuracy: ', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall: ', recall_score(y_test, y_pred))
print('F1 Score: ', f1_score(y_test, y_pred))
print('ROC AUC: ', roc_auc_score(y_test, y_prob))

# Use F1 for imbalanced classes
# Use AUC-ROC for ranking quality
```